1. Data Requirements Analysis

1.1 Introduction

The proposed project "API Test Automation Framework for E-Commerce using Karate" focuses on automating testing of core e-commerce flows such as authentication, product catalog, shopping cart, checkout, and orders using available REST APIs. To design effective automated tests, it is essential to clearly define the data required by each module, how that data is structured, and how it flows between the framework and the APIs under test.

OssJavaHub trial e-commerce APIs such as **testJSON** and **testStore API** provide realistic cart, product, and user data which will be used as the primary external data sources for this framework

1.2 Data Sources

The framework interacts with the following categories of data sources:

- **REST APIs**
    - **testJSON** Carts API, for realistic shopping cart data (cart id, products, quantities, totals).
    - **testStore** API, for products and cart operations (user carts, date-based queries, add cart).
- **Local test data files**
    - JSON files defining request payloads and expected responses for selected APIs.
    - CSV/Excel or Scenario Outline tables in feature files for data-driven tests.
- **Configuration and environment data**
    - Karate configuration (karate-config) holding base URLs (**testJSON** , **testStore**), default headers, environment names, and authentication-related parameters if required later.

These sources together provide all data needed for test execution, validation, and reporting.

1.3 Identification of Data Entities

The main logical data entities are:

1.3.1 User and Authentication Data

Although **testJSON** and TestStore carts are accessible without complex login flows, the framework is designed to support authentication data for completeness and future extensibility.

Typical fields:

- userId, username, password (*******), token, sessionId.

Copyright – Proprietary – OSS Java Hub

Usage:

- Used in test cases that simulate user-specific carts (e.g., /carts/user/{userId}), and in potential secured endpoints if added later.

1.3.2 Product Catalog Data

Product data is primarily taken from **TestStore** or **testJSON** product endpoints.

Typical fields:

- id, title/name, description, category, price, rating, stock.

Usage:

- Used in search/filter scenarios and as items added to carts, enabling validation of product fields in API responses.

1.3.3 Shopping Cart Data

Cart data is central to the project and is obtained from **testJSON** and **TestStore** cart endpoints.

Example **testJSON** cart fields:

- id: Cart identifier.
- products: Array of products in the cart, each with id, title, price, quantity, total, discountPercentage, discountedTotal.
- Aggregate fields: total, discountedTotal, userId, totalProducts, totalQuantity.

Example TestStore cart fields:

- id, userId, date, products: [{productId, quantity}].

Usage:

- Add cart scenarios (/carts/add on **testJSON** or POST /carts on TestStore).
- Update cart scenarios (PUT/PATCH cart id).
- Retrieve and validate user carts (GET /carts/user/{userId} or similar).

1.3.4 Reporting and Execution Data

During execution, Karate generates additional data such as request logs, response bodies, assertion results, and execution timestamps.

Fields:

- Test name, scenario name, status (pass/fail), request URL, HTTP method, response status, response time, error messages.

These are stored in HTML and JUnit XML reports and are used for analysis and debugging.

Copyright – Proprietary – OSS Java Hub

1.4 Data Structures and Storage

- **JSON documents**:
  Request bodies and expected response fragments are stored as JSON files or inline JSON within Karate feature files.
- **Tabular data (CSV/Excel/Scenario Outline)**:
  Parameter sets such as different userId values, product IDs, or quantity combinations are represented in tables for data-driven tests.
- **Configuration files**:
  Base URLs
  like https://devjson.k8s.glb.ossjavahub.com and https://devapi.k8s.glb.ossjavahub.com and common headers are maintained in the Karate config script, enabling environment-independent test execution.
- **Version control**:
  All data artifacts are stored in a Git repository along with test code, which ensures change tracking and rollback.

1.5 Functional Data Requirements

Key functional data requirements include:

- The framework shall send requests to cart endpoints such as GET /carts, GET /carts/{id}, GET /carts/user/{userId}, POST /carts/add or POST /carts, and validate response fields and status codes.
- The framework shall validate the correctness of totals, discounted totals, and quantities for each cart against the product list returned by the API.
- The framework shall perform data-driven tests by varying userId, productId, and quantity values to verify different cart compositions.
- The framework shall verify that cart data remains consistent across retrieval operations (e.g., values from POST/PUT responses must match subsequent GET responses).

These requirements ensure that the data handled by the framework is accurate, consistent, and aligned with e-commerce business rules.

1.6 Non-Functional Data Requirements

- **Data consistency**: IDs and aggregate values like total and totalQuantity must remain consistent throughout a test scenario, even when multiple endpoints are called.
- **Security and privacy**: Real customer data is not used; only dummy data from OSS JAVA Hub APIs and artificially created credentials are used to avoid privacy concerns.
- **Performance metrics**: Response time, status codes, and error rates for cart endpoints must be captured during load and stress tests using Karate's Gatling integration.

Copyright – Proprietary – OSS Java Hub

- **Scalability**: The framework must support adding new endpoints (e.g., orders, payments) without redesigning existing data structures.

1.7 Data Flow

At a high level, the data flow is as follows:

1. Karate loads configuration (base URL, headers) and test data (user ids, product ids, quantities).
2. Karate scenarios construct HTTP requests (GET/POST/PUT/DELETE) to **testJSON** / TestStore endpoints.
3. The API returns JSON responses containing cart, product, and user data.
4. Karate assertions validate field values, structural constraints, and status codes; results are written into reports.

A simple data-flow diagram can later be added showing: Test Data → Karate → API → Responses → Reports.

Copyright – Proprietary – OSS Java Hub

2. System Study

2.1 Introduction

The System Study examines the current practices for e-commerce API testing and contrasts them with the proposed Karate-based automation framework that uses APIs like **testJSON** and TestStore for realistic shopping cart scenarios. This helps justify the need for the framework and clarifies how it will operate in practice.

2.2 Existing System

In many educational and small-industry environments, e-commerce API testing is performed using manual tools such as Postman or browser plugins without a dedicated automation framework.

Common characteristics:

- Test cases are documented in spreadsheets or simple documents and executed manually by sending requests and checking responses.
- There is limited integration with CI/CD pipelines, so regression testing is not consistently automated.
- Performance and load testing of cart operations is rarely performed due to tool complexity.

This approach leads to high effort, low repeatability, and increased risk of missed defects in critical flows such as cart and checkout.

2.3 Limitations of Existing System

The limitations of the existing manual or semi-automated process are:

- **High manual effort**: Re-running the same cart and product tests for every change is time-consuming and error-prone.
- **Poor coverage**: Not all combinations of users, products, and quantities are tested regularly, leaving potential defects undetected.
- **Lack of integration**: API tests are not part of CI/CD pipelines, so feedback on failures arrives late in the development cycle.
- **Limited non-functional testing**: Response time and concurrency effects on cart APIs are seldom measured systematically.

These issues motivate the development of a dedicated, automation-first framework.

Copyright – Proprietary – OSS Java Hub

2.4 Proposed System

The proposed system is a **Karate-based API Test Automation Framework** targeting e-commerce carts and related operations using APIs.

Key features:

- Uses Karate's BDD-style .feature files to define readable test scenarios for cart and product endpoints.
- Integrates with Maven and JUnit for execution from IDE as well as from CI/CD tools like Jenkins or GitHub Actions.
- Interacts with APIs such as **testJSON** (https://devjson.k8s.glb.ossjavahub.com /carts) and TestStore (https://devapi.k8s.glb.ossjavahub.com /carts) to simulate realistic shopping-cart behaviours.
- Supports both functional and performance tests, with potential extension to mocking if needed.

This system reduces manual effort, improves coverage, and enables continuous and repeatable API testing.

3.5 System Objectives

The main objectives of the proposed framework are:

- To design and implement a reusable API test automation framework for core e-commerce/Store operations such as product listing and shopping cart management.
- To consume available e-commerce/Store APIs (**testJSON** , TestStore) as realistic backends without requiring a private server.
- To support data-driven testing by varying user, product, and quantity combinations for cart scenarios.
- To integrate automated tests into CI/CD pipelines to enable continuous validation.
- To provide detailed, human-readable reports that assist in diagnosing failures and analyzing response data.

3.6 System Architecture

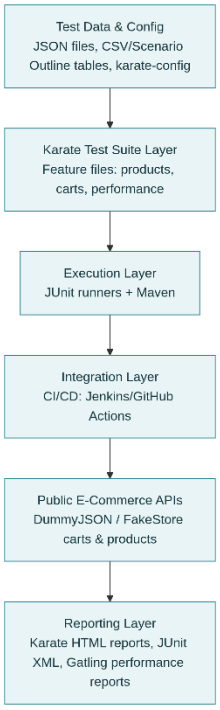
The system follows a layered architecture:

- **Test Suite Layer**
    - Karate .feature files grouped by functionality (products, carts, user carts, performance tests).
- **Execution Layer**
    - JUnit test runners and Maven configuration, enabling command-line and IDE-based execution.

Copyright – Proprietary – OSS Java Hub

- **Integration Layer**
    - Jenkins / GitHub Actions workflows that run the test suites on code commits or on schedule.
- **Reporting Layer**
    - Karate HTML reports and JUnit XML outputs summarizing requests, responses, and assertion results.

A block diagram can show these layers and their interactions with the APIs.



3.7 System Modules

Main modules within the framework are:

- **Product Module**
    - Tests product listing, filtering, and retrieval using endpoints like /products and /products/{id} on **testJSON** or TestStore.
- **Cart Module**
    - Covers scenarios for listing carts, retrieving a specific cart, retrieving carts by user, and adding/updating carts using **testJSON** and TestStore cart endpoints.
- **Data-Driven Module**

    - Manages test data sets for users, products, and quantities, enabling Scenario Outline-based execution.

- **Performance Module**
    - Executes load and stress tests on selected cart endpoints using Karate's Gatling integration to collect response time and throughput metrics.

These modules together form a comprehensive and extensible automation solution for e-commerce/Store API testing provided to the OSSJAVAHub customers.

Copyright – Proprietary – OSS Java Hub